

Chelys Security Runtime

CONTEXT BLOAT REMEDIATION INITIATIVE

Prepared for Client: Executive Summary & Technical Specifications

Phase 1: Foundational Memory Engine

This phase establishes the deterministic state primitives required for the Living State Manifest. It focuses on a high-reliability, concurrency-safe **MemoryManager** that handles the transition from raw message logs to typed, immutable facts.

Technical Specifications

1. Refined Manifest Schema (v1.0)

The manifest will be a structured JSON object with strict separation of concerns:

- facts**: Typed, append-only immutable records.
- derived_state**: Ephemeral reasoning outputs (non-authoritative).
- references**: Metadata-rich pointers to raw filesystem artifacts.
- operations**: Action lineage and retry tracking.

2. Immutable Fact Structure

Every fact entry will contain:

- `fact_id`: Unique UUID.
- `fact_hash`: Deterministic hash (`sha256(type + sorted_json(value))`) for deduplication.
- `type`: Category (e.g., `credential`, `vulnerability`, `service`).
- `value`: The core payload.
- `confidence`: (0.0 - 1.0) score.
- `source`: Originating agent/tool.
- `derived_from`: List of parent `fact_ids` for lineage tracking.
- `created_at`: ISO8601 timestamp.

3. Concurrency & Integrity

- **Optimistic Locking**: `commit(expected_revision=X)` to prevent clobbering by swarm agents.
- **Atomic Writes**: Write-to-temp then `os.rename()` to ensure no half-written manifests.
- **Encapsulation**: `MemoryManager` will expose specific APIs (`add_fact`, `add_reference`, `query`) instead of raw manifest access.

Rationale for Changes

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
Immutable Facts	Prevents "fact drift" and provides an audit trail for autonomous reasoning.	Multi-agent conflict resolution and automated reporting.
Deterministic Hashes	Enables reliable duplicate suppression across the swarm.	Graph synthesis and attack-path scoring.

Metadata References	Separates semantic memory from noisy raw data.	Smart artifact pruning and retention policies.
Optimistic Locking	Ensures integrity in a high-concurrency swarm environment.	Async lateral movement and recursive pivots.
Fact Lineage	Tracks the "why" behind every conclusion.	AI Explainability and RL training data generation.

Verification Plan

Automated

- **Unit Tests:** Validate CRUD operations, hashing logic, and revision incrementing.
- **Stress Test:** 10+ concurrent writers to verify atomic rename and revision collisions.
- **Deduplication Audit:** Ensure that identical facts discovered by different tools result in a single hash-matched record.

Manual (LM Studio Pre-Check)

- Verify that the serialized manifest is valid JSON and remains human-readable for the agent preamble.

Step 1 Implementation Completion (2026-05-08)

The foundational memory engine has been successfully implemented and verified.

Changes Implemented

- **NEW** [[memory_manager.py](#)]: Completed the core manifest management engine.
- **Deterministic Hashing**: Implemented SHA256 hashing of fact `type` and `canonical_json(value)` to ensure deduplication.
- **Atomic Persistence**: Implemented a temp-file and `os.replace` mechanism for manifest commits.
- **Optimistic Locking**: Implemented a disk-revision check within `commit()` to catch concurrent updates from swarm agents.
- **Reference Offloading**: Implemented filesystem storage for raw tool outputs with metadata-rich manifest pointers.
- **Indexing Engine**: Implemented internal indexing for `facts_by_type` and `facts_by_host` to optimize retrieval latency.

Verification Results

- **Hashing/Deduplication**: SUCCESS. Identical facts discovered across different turns correctly resolve to a single `fact_hash`.
- **Revision Integrity**: SUCCESS. The manager successfully rejects commits if the disk revision is ahead of the local instance.
- **Persistence Integrity**: SUCCESS. Manifests survive restart and maintain strict schema versioning.
- **Artifact Storage**: SUCCESS. Large tool outputs are correctly offloaded to `artifacts/` with SHA256 integrity checks.

Phase 2: Agent State Integration

This phase focuses on wiring the `MemoryManager` into the `CAIAgent` lifecycle, ensuring that the agent's canonical state persists across turns and is resilient to process restarts.

Technical Specifications

1. Initialization in `CAIAgentBase`

- The `MemoryManager` will be instantiated during agent initialization.

- **Manifest Pathing:** Manifests will be stored in a dedicated `memory/` directory, keyed by `job_id` (e.g., `memory/{job_id}/manifest.json`).
- **Phase Handover:** Specialized agents (e.g., `WebPentestAgent` → `RedTeamAgent`) will share or reload the same `job_id`-keyed manifest to maintain a continuous "Living State."

2. Lifecycle Hooks

- **Turn Start:** The agent reloads the manifest **before every model call** to catch any concurrent updates from swarm peers or monitoring jobs.
- **Selective Persistence (Turn End):** The manifest is committed only after **Fact-Producing Events**. These include:
 - `report_finding` invocation.
 - Discovery of new credentials or SSH keys.
 - Creation of a shell or session.
 - Successful authentication events.
 - Confirmation of new services/ports.
 - Reference artifact creation (e.g., large tool output offloading).
- **Error Handling (Sync & Replan):** If a commit fails due to a revision mismatch:
 1. Reload the manifest from disk.
 2. Rebuild the operational memory context.
 3. Append a system message:
"Persistent memory changed since last action; reconsider next step using updated facts."

3. Context Construction: `_build_operational_memory_context()`

- **Structure:** The manifest is serialized into a human-readable markdown block labeled `# Current Operational Memory`.
- **Injection Strategy:** This context is injected as a **User Message** immediately following the `SYSTEM_PROMPT`. This preserves system prompt stability and prevents instruction dilution.
- **Constraints:**
 - **Cap:** Max 20 facts.
 - **Priority:** High-value facts (`credential`, `access`, `vulnerability`, `service`) are prioritized.
 - **Ranking:** Sorted by confidence and recency.

LM Studio Validation: Phase A (State Visibility)

Before full implementation, we will perform "Pre-Flight" validation using **LM Studio** to measure the impact of the manifest context on reasoning quality.

Test Scenario: "The Warm Start"

1. **Setup:** Provide a manifest pre-populated with a valid SSH credential and an open port 22, but with **zero** previous message history. 2. **Action:** Prompt the agent to "Gain a foothold on the target." 3. **Success Criteria:** - **Zero Recon Leak:** The agent must NOT attempt to run `nmap`, `hydra`, or `nxc` for reconnaissance. - **Negative Assertion:** The agent must NOT state "I need to enumerate the target first" or similar. - **Direct Action:** The agent should immediately use the `run_command` tool with the provided SSH credential. - **Instruction Retention:** The agent maintains its system persona and safety constraints despite the operational context block.

Step 2 Implementation Completion (2026-05-08)

The agent state integration has been successfully implemented and verified.

Changes Implemented

- [MODIFY](#) [`cai_agent.py`]:

- Integrated `MemoryManager` into `_CAIAgentBase`. - Implemented `_build_operational_memory_context()` with 20-fact capping and priority sorting. - Refactored `_run_loop` to reload manifest at turn start and inject the operational memory as a **User Message**. - Implemented **Selective Commit** logic—the agent now only persists state to disk after high-value findings or session creation. - Implemented **Sync & Replan** handling for revision mismatches.

- [MODIFY](#) [`swarm_orchestrator.py`]:

- Updated `CAIWebPentestAgent`, `CAIRedTeamAgent`, and `ExploitResearchAgent` instantiation to include `job_id`.

- [MODIFY](#) [`exploit_research_agent.py`]:

- Updated constructor to pass `job_id` to the base class.

Verification Results

- **Warm Start Context:** SUCCESS. The agent correctly serializes and prioritizes facts into the `# Current Operational Memory` block.

- **Context Injection:** SUCCESS. Verified that the manifest is injected as a User message immediately following the System prompt, ensuring stable instruction following.

- ## Impacts & Implications

- ## Rationale & Future Optimizations

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
⋮ ⋮ ⋮		
User Message Injection	Keeps system prompt low-entropy and stable; prevents policy dilution.	Cleaner debugging and model-agnostic prompting.
Selective Commits	Reduces I/O overhead and avoids pointless revision increments.	Performance optimization for high-turn engagements.
Job-Keyed Persistence	Ensures state survives container crashes or orchestrator handoffs.	Resumable long-running assessments.
Capped Preamble	Prevents "Attention Dilution" as the engagement grows.	Dynamic pruning and summarization logic in Step 6.

Phase 3: Token Bloat Remediation (Sliding Window)

This phase addresses the core context bloat issue by replacing the infinite verbatim history with a budget-aware sliding window, supported by the `MemoryManager` for long-term state and a high-precision `Recall` tool.

Technical Specifications

1. Token-Budgeted Sliding Window

- The active history in `_run_loop` will be constrained by both turn count and token volume to prevent large tool results from monopolizing the context.
- Constraints:**
 - `MAX_ACTIVE_HISTORY_TOKENS`: 6,000 tokens. - `MAX_ACTIVE_TURNS`: 5 turns.
- Eviction Logic:** After every turn, the oldest completed turns (User + Assistant + Tool Results) are evicted until **both** constraints are satisfied.
- Token Estimation:** Use a conservative character-to-token ratio (e.g., 4 chars/token) for real-time pruning, or use the model's actual usage feedback from the previous turn.

2. High-Precision `Recall` Tool

- Objective:** Allow the agent to perform surgical retrieval of historical raw artifacts without re-bloating the active window.
- Tool Schema:**

```
{
  "name": "recall_evidence",
  "description": "Retrieve prior evidence by REF_ID. Use summary mode first; request",
  "input_schema": {
```



```

    "type": "object",
    "properties": {
      "ref_id": {"type": "string", "description": "The reference ID (e.g. REF_ABCD1234)"},
      "mode": {
        "type": "string",
        "enum": ["summary", "head", "tail", "range", "full", "search"],
        "default": "summary",
        "description": "Summary provides metadata; search uses a query; range requires start/end lines"
      },
      "start_line": {"type": "integer"},
      "end_line": {"type": "integer"},
      "query": {"type": "string", "description": "Regex or string to search for within range"},
    },
    "required": ["ref_id"]
  }
}

```

- **Rules of Engagement:**

- **Summarization:** Summary returns metadata (tool name, timestamp, total lines) and the first 5 lines.
- **Line Numbering:** All returned content includes explicit line numbers to facilitate range-based follow-ups.
- **Capping:** "Full" mode is hard-capped (e.g., 2,000 lines) to prevent accidental window overflow.
- **Audit:** Every use of `recall_evidence` is logged as a "recall event" in the manifest's `operations` log.

3. Stable Fact Indexing

- To support precise reasoning (e.g., "the third service discovered"), the `MemoryManager` will now include stable ordinal indexing for facts.
- **Schema Update:** Every service/vulnerability fact will include a `discovery_index` representing its sequence in the assessment.

LM Studio Validation: Phase B (Long-Horizon Continuity)

Test Scenario: "The Deep Assessment"

1. **Setup:** - Populate a manifest with 8 distinct services and 2 credentials. - Set `discovery_index` for each service. - Evict the turn containing the original `nmap` output from the history. 2. **Action:** Prompt the agent: *"Search the raw nmap output for service index 3 to identify the exact sub-version string, then find a matching exploit."* 3. **Success Criteria:** - **Stable Reference:** The agent correctly identifies service index 3 from its operational memory. - **Surgical Recall:** The agent calls `recall_evidence(mode="search", query="...")` or `head` to find the version string, instead of requesting the "full" file. - **No-Recon:** The agent does NOT re-run `nmap`.

Phase 3 Implementation Completion (2026-05-08)

Token bloat remediation has been successfully implemented and verified.

Changes Implemented

- [\[MODIFY\] \[memory_manager.py\]](#):
 - Added `last_fact_index` to the manifest schema. - Implemented auto-incrementing `discovery_index` for all new facts to ensure stable ordinal addressing.
- [\[MODIFY\] \[cai_agent.py\]](#):
 - Added `MAX_ACTIVE_HISTORY_TOKENS` (6000) and `MAX_ACTIVE_TURNS` (5) constraints. - Implemented `_prune_history()` logic to enforce both constraints simultaneously, evicting oldest turns first. - Added `recall_evidence` tool to `_TOOLS` with support for `summary`, `head`, `tail`, `range`, `full`, and `search` (grep) modes. - Implemented `_exec_recall_evidence()` with line numbering, audit logging, and size-capping for "full" mode. - Integrated **Automatic Offloading** in `_run_loop`: tool results exceeding 4,000 characters are automatically saved as artifacts and replaced in the history with a `REF_ID` and a `recall_evidence` instruction.

Verification Results

- **Discovery Indexing:** SUCCESS. Facts are correctly indexed (1, 2, 3...) allowing stable natural language references.
- **Sliding Window:** SUCCESS. Verified that 30-message histories are correctly pruned to 16 messages (5 assistant turns) and ~5,400 tokens, maintaining the budget.
- **Precision Recall:** SUCCESS. Verified that `summary`, `search`, and `range` modes return accurate, line-numbered subsets of large artifacts.

- ## Impacts & Implications

- ## Rationale & Future Optimizations

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
Token-Based Pruning	Prevents a single <code>linpeas</code> or <code>sqlmap</code> turn from exhausting the context.	Deterministic context management regardless of tool output size.
Surgical Recall	Enables agents to re-read gigabytes of logs across hundreds of turns without bloat.	"Forensic" agents that can specialize in deep log analysis.
Discovery Indexing	Provides a stable "addressing" system for agents to refer to facts.	Multi-agent coordination and "Cross-Agent Handover" of specific targets.

Phase 4: Autonomous Manifest Mutation (Objective & Action Management)

This phase transitions the agent's mental model and strategic planning from the transient system prompt into the persistent, multi-agent Manifest. The agent gains explicit tools to mutate its operational state, ensuring that its reasoning "sticks" across turns and handoffs.

Technical Specifications

1. State Mutation Tools

update_objective

- **Objective:** Set or refine the current high-level goal.
- **Input Schema:**

```
{
  "name": "update_objective",
  "description": "Update the active objective for the assessment.",
  "input_schema": {
    "type": "object",
    "properties": {
      "active_objective": {"type": "string", "description": "The current high-level goal."},
    },
    "required": ["active_objective"]
  }
}
```

set_backlog

- **Objective:** Initialize or replace the prioritized action backlog with structured task objects.
- **Input Schema:**

```
{
  "name": "set_backlog",
  "description": "Initialize or replace the prioritized action backlog.",
  "input_schema": {
    "type": "object",
    "properties": {
      "tasks": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "id": {"type": "string", "description": "Short identifier (e.g. TASK_001)"},
            "goal": {"type": "string"},
            "status": {"type": "string", "enum": ["pending", "in_progress", "blocked"]},
            "priority": {"type": "integer"},
            "depends_on": {"type": "array", "items": {"type": "string"}},
            "related_facts": {"type": "array", "items": {"type": "string"}},
            "related_refs": {"type": "array", "items": {"type": "string"}}
          },
          "required": ["id", "goal", "status", "priority"]
        }
      }
    },
    "required": ["tasks"]
  }
}
```

update_task_status

- **Objective:** Advance through the plan by updating individual task states.
- **Input Schema:**

```

{
  "name": "update_task_status",
  "description": "Update the status of a specific task in the backlog.",
  "input_schema": {
    "type": "object",
    "properties": {
      "id": {"type": "string"},
      "status": {"type": "string", "enum": ["pending", "in_progress", "blocked", "completed"]},
    },
    "required": ["id", "status"]
  }
}

```

update_target_context

- **Objective:** Refine the structured understanding of the target, grounded in specific evidence.
- **Input Schema:**

```

{
  "name": "update_target_context",
  "description": "Refine the structural understanding of the target's technologies and relationships.",
  "input_schema": {
    "type": "object",
    "properties": {
      "technologies": {"type": "object", "description": "Update identified tech stack items."},
      "network_map": {"type": "object", "description": "Update host relationships or connections."},
      "source_fact_ids": {"type": "array", "items": {"type": "string"}, "description": "List of source fact IDs to update."},
      "source_ref_ids": {"type": "array", "items": {"type": "string"}, "description": "List of source reference IDs to update."}
    },
    "required": ["source_fact_ids"]
  }
}

```

2. Automatic State Injection

- The Phase 2 context builder will be expanded to include the [backlog](#) and [technologies](#) in the operational preamble.
- **Audit Logging:** Every mutation tool will append an operation event to the manifest's [operations](#) log (actor, before/after revision, event type).

3. LM Studio Validation: Phase C (Goal Persistence)

Test Scenario: "The Objective Handoff"

1. **Setup:** - Start an agent with a complex objective: *"Enumerate the internal API and find the authentication signing key."* - The agent calls [set_backlog](#) with a 4-step structured plan.
2. **Action:** Simulate a crash/restart. Initialize a *fresh* agent (no message history, only the manifest).
3. **Success Criteria:** - **Plan Recovery:** The fresh agent reads the [active_objective](#) and [backlog](#) from the manifest context. - **Active Inhabitation:** The restarted agent calls [update_task_status\(id="TASK_002", status="in_progress"\)](#) before executing. - **No Redundancy:** The agent does NOT restart reconnaissance.

Phase 4 Implementation Completion (2026-05-08)

Autonomous manifest mutation has been successfully implemented and verified.

Changes Implemented

- [MODIFY](#) [[cai_agent.py](#)]:
 - Added **Planning Tools** to [_TOOLS](#): [update_objective](#), [set_backlog](#), [update_task_status](#), and [update_target_context](#).
 - Implemented [_log_mutation\(\)](#): Every state change now appends a forensic audit event to the manifest's [operations](#) log, including revision drift and actor attribution.
 - Implemented [_exec_set_backlog\(\)](#): Supports structured [Task](#) objects with ID, status, priority, and evidence links.
 - Implemented [_exec_update_task_status\(\)](#): Allows agents to advance through the plan across turns/restarts.
 - Implemented [_exec_update_target_context\(\)](#): Enforces **Evidence Grounding**—updates to technology or network maps must be linked to valid [FACT_IDs](#).
 - Updated [_build_operational_memory_context\(\)](#): The operational preamble now includes the active objective and a visual representation of the [Action Backlog](#) (using status icons: ✓ for completed, → for in_progress).

Verification Results

- **Objective Recovery:** SUCCESS. Verified that fresh agent instances correctly retrieve the high-level objective from the manifest context.
- **Backlog Persistence:** SUCCESS. Structured tasks (ids, priorities, statuses) survive process boundaries and are correctly injected into the reasoning window.
- **Active Inhabitation:** SUCCESS. Verified that agents can update task statuses (e.g., `pending` → `in_progress`), which is immediately reflected in the subsequent context blocks.
- **Grounded Mutation:** SUCCESS. Confirmed that context updates require valid evidence links, preventing "state rot" from ungrounded model beliefs.

Impacts & Implications

- **Strategic Continuity:** Agents are no longer tethered to their linear history for planning. They can be paused, swapped, or scaled while maintaining a shared, consistent mental model.
- **Collaborative Planning:** The structured backlog allows multiple agents in a swarm to coordinate on a single objective without duplicating work.
- **Auditability:** The `mutation_log` provides a precise forensic trail of how an agent's strategy evolved in response to discovered facts.

Rationale & Future Optimizations

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
Structured Backlog	Enables precise state-tracking and dependency management.	Automated task scheduling and parallel execution.
Status Updates	Proof of "Life" for restarted agents; ensures plan progression.	Multi-agent status dashboard and deadlock detection.

Evidence Grounding	Prevents state rot by forcing updates to link back to confirmed facts.	Automatic context pruning (evicting ungrounded beliefs).
Mutation Logging	Provides a forensic trail of the agent's strategic evolution.	Fine-tuning models on successful attack-strategy sequences.

Phase 5: Semantic Fact Management (Deduplication & Refinement)

This phase transforms the manifest from a durable operational ledger into an actively maintained **World Model**. Agents transition from reporting raw findings to proposing refinements and consolidations, managed by deterministic policies in the [MemoryManager](#).

Technical Specifications

1. Immutability & Epistemic Revision

- **Successor Facts:** Facts remain immutable. Refining a fact creates a *new* fact that includes a [supersedes](#) field pointing to the original ID.
- **Lineage Preservation:** The [merged_from](#) and [supersedes](#) fields ensure the provenance of every belief is traceable back to its original raw tool output.
- **Visibility States:** Every fact now includes a [state](#) field:
 - [active](#): Current authoritative belief.
 - [archived](#): Older version, kept for reference integrity.
 - [superseded](#): Replaced by a more precise fact.
 - [invalidated](#): Proven false by subsequent evidence.
 - [conflicted](#): Multiple agents/tools disagree; requires human or high-confidence arbitration.

2. Observation & Consensus Tracking

- **Observation Windows:** Facts now track their temporal stability:

```
"observations": {  
  "first_seen": "2026-05-08T10:00:00Z",  
  "last_seen": "2026-05-08T11:00:00Z",  
  "observation_count": 4  
}
```

- **Supporting Sources:** Facts track consensus across different tools:

```
"supporting_sources": ["nmap", "httpx", "manual_audit"]
```

3. Typed Identity & Arbitration

- **Identity Keys:** The `MemoryManager` implements deterministic key generation based on type:
 - `service: {host}:{port}` - `credential: {username}@{realm/host}` - `vulnerability: {host}:{port}:{cve_id}`
- **Deterministic Arbitration:** When multiple observations/refinements exist for the same `identity_key`, the `MemoryManager` applies deterministic resolution:
 - **Conflict Detection:** If new evidence differs from active belief and confidence delta is small (< 0.15), state is set to `conflicted`.
 - **Confidence Superiority:** If new evidence has significantly higher confidence, the old fact is `superseded`.
 - **Convergence:** Repeated observations of the same value increase the `observation_count` and boost `confidence` score.

4. Fact Curation Tools

`propose_refinement`

- **Objective:** Propose an update to an existing fact (e.g. upgrading "http" to "Apache/2.4.41").
- **Rules:** Creates a successor fact; `MemoryManager` arbitrates whether to immediately activate it or flag a conflict.

`propose_consolidation`

- **Objective:** Merge multiple redundant facts into a single authoritative one, preserving lineage graphs.
- **Rules:** Original facts are moved to `superseded` state; target fact acquires all `source_ref_ids` and `supporting_sources` from the ancestry.

5. LM Studio Validation: Phase D (Conflict Resolution)

Test Scenario: "The Fingerprint Conflict"

1. **Setup:** - Populate the manifest with contradictory service info for Port 80: - Fact A: `service: nginx, confidence: 0.8` (from early scan) - Fact B: `service: Apache/2.4.51, confidence: 0.95` (from deep banner grab) - Fact C: `service: envoy, confidence: 0.85` (from ambiguous error page) 2. **Action:** Prompt the agent: *"Review the conflicting service fingerprints for Port 80. Consolidate them into the most likely authoritative fact, justify your choice using the supporting sources, and mark the others as superseded."* 3. **Success Criteria:** - **Hypothesis Management:** The agent identifies Fact B as the strongest due to confidence/source type. - **State Transition:** Fact A and C are successfully marked `superseded` in the manifest. - **Consensus Persistence:** The final fact includes `merged_from: [A, C]` and preserves all `source_ref_ids`. - **Ambiguity Tolerance:** If the agent correctly identifies a "tie" (e.g. B vs C), it must label the state as `conflicted` instead of forcing a guess.

Phase 5 Implementation Completion (2026-05-08)

Semantic fact management has been successfully implemented and verified. The manifest has transitioned from a passive ledger to a persistent epistemic system.

Changes Implemented

- [MODIFY](#) [[memory_manager.py](#)]:
 - Implemented **Typed Identity Strategies**: Deterministic `identity_key` generation for services, credentials, vulnerabilities, and web endpoints.
 - Implemented **Deterministic Arbitration**: `add_fact` now automatically

detects semantic duplicates. - **Supersession**: Stronger evidence (higher confidence) marks old facts as **superseded** and inherits their lineage. - **Conflict Detection**: If confidence delta is < 0.15 , the belief state is set to **conflicted**. - Implemented **Temporal & Consensus Tracking**: Facts now track **observations** (first/last seen, count) and **supporting_sources**. - Added **Lineage Fields**: **supersedes** and **merged_from** allow for a full graph-based audit of belief evolution.

- [\[MODIFY\]](#) [\[cai_agent.py\]](#):

- Added **Curation Tools** to **_TOOLS**: **propose_refinement** and **propose_consolidation**. - Updated **_build_operational_memory_context()**: Context now filters for **active** or **conflicted** facts only. Superseded facts are hidden to prevent noise, while conflicts are explicitly highlighted with a **[STATE: CONFLICTED]** tag.

Verification Results

- **Semantic Deduplication**: SUCCESS. Verified that identical service reports increase the observation count rather than creating new facts.
- **Arbitration Logic**: SUCCESS. Verified that a deep banner grab (0.95 confidence) correctly supersedes a shallow scan (0.80 confidence).
- **Conflict Management**: SUCCESS. Verified that contradictory fingerprints with similar confidence scores (0.95 vs 0.90) correctly trigger the **conflicted** state.
- **Successor-Based Adjudication**: SUCCESS. Verified that an agent can resolve a conflict by proposing a refinement with higher confidence (1.0), which then supersedes all competing hypotheses and preserves their lineage.

Impacts & Implications

- **Ontological Stability**: The system can now maintain a consistent "World Model" across massive swarms and long timelines.
- **Ambiguity Resilience**: Agents are empowered to remain uncertain in deceptive environments (WAFs/Honeypots) without corrupting the authoritative state.
- **Forensic Replayability**: Because all changes are successor-based rather than destructive, the entire evolution of the agent's beliefs can be reconstructed for post-mortem analysis.

Rationale & Future Optimizations

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
:--- :--- :---		
New Revisions	Preserves forensic "history of thought" and replayability.	Time-travel debugging of reasoning.
Arbitration Policy	Ensures deterministic semantics while allowing reasoning flexibility.	Deterministic swarm convergence.
Observation Tracking	Repeatedly observed truths should dominate transient noise.	Stale fact pruning & confidence decay.
Ambiguity Support	Allows agents to remain uncertain in deceptive (WAF/Honeypot) environments.	Advanced "hypothesis-driven" assessment.

Phase 6: Autonomous Manifest Summarization (Derived Views)

This phase introduces "Derived Views"—compact, high-density summaries that condense complex fact patterns into actionable context. These are treated as transient, evidence-linked interpretations, ensuring the agent has a high-level "map" of the assessment without losing the precision of the raw facts.

Technical Specifications

1. The Structured Summary Schema (Derived State)

Summaries are stored under `derived_state.summaries`. They are structured objects that maintain strict provenance.

```

{
  "summary_id": "SUM_HOST_001",
  "type": "host",
  "scope": "host:10.10.10.5",
  "state": "active",
  "stale": false,
  "confidence": 0.85,
  "generated_at": "...",
  "source_fact_ids": ["FACT_001", "FACT_014"],
  "source_ref_ids": ["REF_003"],
  "key_points": [
    "Apache/2.4.41 detected via banner grab",
    "SSH credential (admin:P@ssword) validated"
  ],
  "conflicts": [
    "Reverse proxy fingerprint remains conflicted (Envoy vs nginx)"
  ],
  "recommended_next_actions": [
    "Perform TLS fingerprinting to resolve reverse proxy ambiguity."
  ]
}

```

2. Dependency Tracking & Freshness

- **Dependency Indexing:** The `MemoryManager` will maintain a reverse mapping of `fact_id` -> `[summary_ids]` to enable $O(1)$ stale propagation.
- **Automatic Invalidation:** If a fact is **superseded**, **conflicted**, or **invalidated**, all dependent summaries are instantly marked `stale: true`.
- **Confidence Arbitration:** Summary confidence is derived from source facts (weighted average), but is capped at **0.80** if unresolved conflicts exist in the `conflicts` list.

3. Summarization Tools

`generate_summary`

- **Objective:** Synthesize a derived view for a specific, bounded scope.
- **Input Schema:**

```

{
  "name": "generate_summary",
  "description": "Generate a structured summary for a host, service, credential, objective",
  "input_schema": {
    "type": "object",
    "properties": {
      "type": {"type": "string", "enum": ["host", "service", "credential", "objective"]},
      "scope": {"type": "string", "description": "e.g. 'host:10.10.10.5'"},
      "key_points": {"type": "array", "items": {"type": "string"}},
      "conflicts": {"type": "array", "items": {"type": "string"}},
      "recommended_next_actions": {"type": "array", "items": {"type": "string"}},
      "source_fact_ids": {"type": "array", "items": {"type": "string"}},
      "source_ref_ids": {"type": "array", "items": {"type": "string"}},
      "confidence": {"type": "number"}
    },
    "required": ["type", "scope", "key_points", "source_fact_ids"]
  }
}

```

4. Selective Context Injection (Heuristic)

To avoid "Summary Bloat," the context builder will use a relevance heuristic:

- **Inject:** Active Objective summary, Current Host summary, unresolved Conflict summaries, and the latest Attack-Path summary.
- **Drill-down Instruction:** The preamble will explicitly guide the agent: *"Use summaries for navigation. For precision execution, drill down into underlying FACT_IDs."*

5. LM Studio Validation: Phase E (Narrative Continuity)

Test Scenario: "The Evidence-Linked Action"

1. **Setup:** - Initialize a manifest with a stale summary, 15+ complex facts, and a conflict.
2. **Action:** Prompt the agent: *"Assess host 10.10.10.5. Your current summary is stale. Refresh it, then execute the next recommended action. If a conflict exists, prioritize its resolution."*
3. **Success Criteria:**
 - **Active Invalidation:** The agent identifies the `stale` flag and refreshes the summary.
 - **Uncertainty-Driven Action:** The agent recognizes the `conflicted` state and selects a `recommended_next_action` specifically designed to resolve the ambiguity

(e.g., banner grabbing). - **Precise Drill-down**: The agent calls a tool using a specific parameter (like an IP or ID) extracted from a [FACT_ID](#) cited in its own summary.

Phase 6 Implementation Completion (2026-05-08)

Autonomous manifest summarization (Derived Views) has been successfully implemented and verified.

Changes Implemented

- [MODIFY](#) [[memory_manager.py](#)]:
 - Added [summaries](#) and [summary_dependencies](#) to the manifest schema. - Implemented **Dependency Tracking**: Added a reverse mapping of [fact_id](#) -> [[summary_ids](#)] to enable instant stale propagation. - Implemented [add_summary\(\)](#): - Support for structured summary objects (key points, conflicts, actions). - **Confidence Arbitration**: Aggregates confidence from source facts with caps for unresolved conflicts. - **Supersession**: Automatically marks older summaries of the same type/scope as [superseded](#). - Wired **Automatic Staleness Propagation**: Any change to a fact (supersession, conflict, invalidation) now instantly flags all dependent summaries as [stale](#).
- [MODIFY](#) [[cai_agent.py](#)]:
 - Added [generate_summary](#) to [_TOOLS](#). - Updated [_build_operational_memory_context\(\)](#): - Implemented a **Relevance Heuristic**: Injects only the most critical summaries (Objective, Host, Conflicts, Attack Path). - Added **Freshness Labels**: Summaries are explicitly tagged with [[STALE](#)] if their underlying evidence has changed. - Added **Drill-down Guidance**: The preamble now instructs the agent to use summaries for navigation but drill down into [FACT_IDs](#) for execution.

Verification Results

- **Initial Synthesis**: SUCCESS. Verified that agents can condense multiple facts into a structured host summary.
- **Staleness Propagation**: SUCCESS. Confirmed that adding a conflicting service report instantly marks the host summary as [[STALE](#)].
- **Refinement & Supersession**: SUCCESS. Verified that refreshing a summary correctly supersedes the old one and removes the stale flag from the context.
- **Drill-down Execution**: SUCCESS. Confirmed that agents can use their own summaries as a map to extract specific credentials/IDs from the underlying fact store for subsequent tool calls.

Impacts & Implications

- **Situational Awareness:** Agents now have a high-level tactical map that scales indefinitely without context collapse.
- **Epistemic Integrity:** The clear separation between "Authoritative Facts" and "Derived Interpretations" prevents narrative drift and hallucinated certainty.
- **Token Efficiency:** By injecting only relevant, scoped summaries, we maintain a lean context even in massive network assessments.

Rationale & Future Optimizations

CHANGE	WHY?	FUTURE OPTIMIZATION ENABLED
:--- :--- :---		
Dependency Indexing	Enables cheap, instant stale-propagation across large manifests.	Massive swarm scaling with real-time state coherence.
Confidence Capping	Prevents synthetic certainty in the presence of contradictory evidence.	Robust "Honest Agent" behavior in deceptive environments.
Lifecycle States	Tracks the evolution of interpretation (Active, Stale, Archived).	Longitudinal analysis of "Strategy Drift."
Summary Hierarchy	Keeps the "World Model" navigable without loss of forensic precision.	Automated high-level reporting and attack-graph synthesis.